

COMPUTER NETWORKS PROJECT

Private Network Service Platform

Phase 1 — Two-Mac Implementation Report

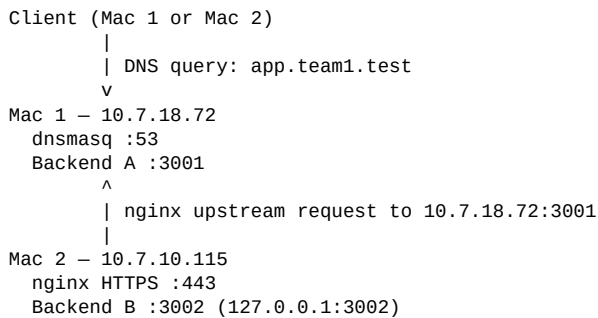
Project detail	Implementation
Student	Mohit Singh
Team setup	Two Macs on the same private Wi-Fi/LAN
Mac 1 — Abhishek	10.7.18.72 — dnsmasq DNS + Backend A (port 3001)
Mac 2 — Mohit	10.7.10.115 — nginx HTTPS/reverse proxy/load balancer (port 443) + Backend B (port 3002)
Private hostname	app.team1.test (also configured: api.team1.test)
Evidence	Terminal outputs and Wireshark captures supplied for the project video

Report organized in the Phase 1 task sequence: network setup, private DNS, backend services, nginx/TLS/load balancing, caching, packet capture, and failure demonstrations.

1. Project Overview

The Private Network Service Platform demonstrates how a client uses private DNS to locate an application endpoint, connects securely to nginx over HTTPS, and is routed to one of two Node/Express backend services. The project uses two Macs on the same Wi-Fi network by combining the DNS and Backend A roles on Mac 1, and combining nginx and Backend B on Mac 2.

1.1 Two-Mac topology



Component	Host / address	Role
Private DNS	Mac 1 — 10.7.18.72:53	Resolves app.team1.test to Mac 2's IP
Backend A	Mac 1 — 10.7.18.72:3001	Returns X-Backend: A and JSON status
nginx	Mac 2 — 10.7.10.115:443	TLS termination, reverse proxy and load balancing
Backend B	Mac 2 — 127.0.0.1:3002	Returns X-Backend: B and JSON status

2. Task A — Private LAN and Host Roles

Both Macs are connected to the same Wi-Fi/LAN. The observed addresses used throughout the implementation are listed below. The two Macs communicate directly by IP for backend tests, while clients use the private hostname for the HTTPS entry point.

Machine	Observed IPv4 address	Services
Mac 1 — Abhishek	10.7.18.72	dnsmasq; Backend A on TCP 3001
Mac 2 — Mohit	10.7.10.115	nginx HTTPS on TCP 443; Backend B on TCP 3002

Note: the supplied outputs verify service reachability at the time of testing. The report does not claim that all possible LAN diagnostics (for example, every ping test) were recorded.

3. Task B — Private DNS with dnsmasq

Mac 1 runs dnsmasq. Mac 2 queries the DNS service at 10.7.18.72. The DNS record for app.team1.test points to Mac 2, where nginx accepts HTTPS requests.

3.1 Verification command

```
dig @10.7.18.72 app.team1.test +short
```

3.2 Observed output

```
10.7.10.115
```

Wireshark evidence shows a DNS query from 10.7.10.115 to 10.7.18.72 for app.team1.test and a DNS response returning A 10.7.10.115. The relevant transport is UDP port 53.

3.3 Wireshark display filter

```
dns
```

```
udp.port == 53
```

4. Task C — Backend Services

The project uses two Node/Express services. Both expose `/api/status` and return JSON identifying the backend. The X-Backend response header distinguishes the selected backend.

4.1 Backend A — Mac 1

```
curl --max-time 3 -i http://10.7.18.72:3001/api/status

HTTP/1.1 200 OK
X-Backend: A
Cache-Control: no-cache
Content-Type: application/json; charset=utf-8
{"backend":"A","status":"ok"}
```

4.2 Backend B — Mac 2

```
curl --max-time 3 -i http://127.0.0.1:3002/api/status

HTTP/1.1 200 OK
X-Backend: B
Cache-Control: no-cache
Content-Type: application/json; charset=utf-8
{"backend":"B","status":"ok"}
```

These direct requests confirm that each backend can answer independently when it is running.

5. Task D — nginx, HTTPS, Reverse Proxy and Load Balancing

nginx runs on Mac 2 and listens on HTTPS port 443. It forwards requests to the upstream backend pool: Backend A at 10.7.18.72:3001 and Backend B at 127.0.0.1:3002. The local tests use `--resolve` to direct the hostname to 127.0.0.1 while retaining `app.team1.test` for TLS hostname validation.

5.1 HTTPS status request

```
curl --max-time 5 -i --cacert "$HOME/certs/ca.crt" \
  --resolve app.team1.test:443:127.0.0.1 \
  https://app.team1.test/api/status

HTTP/1.1 200 OK
Server: nginx/1.31.6
X-Backend: B
Cache-Control: no-cache
{"backend":"B","status":"ok"}
```

The request returned HTTP 200 OK through nginx. Supplying the CA certificate with `--cacert` enabled certificate validation for this request.

5.2 Load-balancing test

```
for i in 1 2 3 4 5 6; do
  curl --max-time 5 -sS --cacert "$HOME/certs/ca.crt" \
    --resolve app.team1.test:443:127.0.0.1 \
    -D - https://app.team1.test/api/status -o /dev/null |
    grep -i '^X-Backend:'
done
```

Observed output when both backends were available:

```
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
```

The alternating X-Backend header is evidence that repeated requests were distributed across both backends in the tested sequence.

6. Task F — HTTP Caching, ETag and Conditional Requests

The `/api/cached` endpoint returns `Cache-Control: max-age=60` and an ETag. A client can send that ETag in the `If-None-Match` request header to ask whether the resource has changed.

6.1 Inspect cache headers

```
curl --max-time 5 -sSI --cacert "$HOME/certs/ca.crt" \
  --resolve app.team1.test:443:127.0.0.1 \
  https://app.team1.test/api/cached

HTTP/1.1 200 OK
X-Backend: A
Cache-Control: max-age=60
ETag: W/"19-3Xq2iFjXnMbnAKEagMsqMcjEGaE"
```

6.2 Conditional request

An initial attempt used the placeholder `PASTE_ETAG_HERE` and correctly returned a full 200 response because it did not contain the actual validator. A later request using the exact ETag returned 304 Not Modified.

```
curl --max-time 5 -i --cacert "$HOME/certs/ca.crt" \
  --resolve app.team1.test:443:127.0.0.1 \
  -H 'If-None-Match: W/"19-3Xq2iFjXnMbnAKEagMsqMcjEGaE"' \
  https://app.team1.test/api/cached

HTTP/1.1 304 Not Modified
X-Backend: B
Cache-Control: max-age=60
ETag: W/"19-3Xq2iFjXnMbnAKEagMsqMcjEGaE"
```

The 304 response indicates that the cached representation is still valid; the server does not need to send the full response body again.

7. Task G — Wireshark Packet Evidence

Wireshark was used to inspect packets on the active Wi-Fi interface `en0`. The submitted screenshots provide evidence for DNS resolution, backend HTTP traffic, and TLS traffic.

Capture / filter	Visible evidence	Interpretation
<code>dns or udp.port == 53</code>	Query for <code>app.team1.test</code> from <code>10.7.10.115</code> to <code>10.7.18.72</code> ; response A <code>10.7.10.115</code>	Private DNS resolution
<code>ip.addr == 10.7.18.72 && ip.addr == 10.7.10.115 && tcp.port == 3002</code>	TCP handshake, GET <code>/api/status</code> , HTTP/1.1 200 OK, JSON response	Mac-to-Mac Backend B connectivity
<code>tcp.port == 443</code>	TCP handshake, TLS 1.3 Client Hello/SNI <code>app.team1.test</code> , Server Hello and Application Data	HTTPS transport and encrypted application traffic

TLS Application Data is encrypted in the capture. The JSON response is demonstrated by the Terminal `curl` output rather than being readable in the TLS packet list.

7.1 Useful Wireshark display filters

```
dns
udp.port == 53
tcp.port == 3001
tcp.port == 3002
tcp.port == 443
ip.addr == 10.7.18.72 && ip.addr == 10.7.10.115 && tcp.port == 443
```

8. Failure Demonstrations and Recovery

The following are the observed failure tests supplied for the report. The sequence below is arranged logically by scenario rather than by the order in which the outputs were pasted.

8.1 Stop Backend A while Backend B remains available

On Mac 1, Backend A was started with `node server.js A 3001` and then stopped with `Ctrl+C`. A repeated six-request test through `nginx` returned Backend B for all six requests. A direct request to Backend B still returned HTTP 200 OK.

```
# Mac 1
node server.js A 3001
# Stop with Ctrl+C

# Mac 2 – repeat the HTTPS request loop
# Observed X-Backend results:
B
B
B
B
B
B
B
```

A direct request to 10.7.18.72:3001 later timed out while Backend A was unavailable, while the direct request to 127.0.0.1:3002 returned 200 OK. After Backend A was restarted, direct access to port 3001 returned HTTP 200 OK with X-Backend: A.

8.2 Stop Backend B

When Backend B was stopped, a direct request to 127.0.0.1:3002 failed to connect. With both upstream backends unavailable, nginx returned 502 Bad Gateway.

```
curl --max-time 3 -i http://127.0.0.1:3002/api/status
# Observed: curl: (7) Failed to connect to 127.0.0.1 port 3002

curl --max-time 5 -i --cacert "$HOME/certs/ca.crt" \
  --resolve app.team1.test:443:127.0.0.1 \
  https://app.team1.test/api/status
# Observed:
HTTP/1.1 502 Bad Gateway
Server: nginx/1.31.6
```

8.3 Restore services and verify

Backend A was restarted on Mac 1 with node server.js A 3001. The supplied later checks show both direct backends returning HTTP 200 OK, and the six-request HTTPS test again alternating between B and A. One conditional-cache request timed out on its first attempt; repeating it with the exact ETag returned 304 Not Modified.

```
# Mac 1
cd ~/cn-backend
node server.js A 3001

# Mac 2 – final checks
curl --max-time 3 -i http://10.7.18.72:3001/api/status
curl --max-time 3 -i http://127.0.0.1:3002/api/status
```

The evidence supplied does not include separate demonstrations for every failure listed in the original Phase 1 guide (wrong DNS server, incorrect DNS record, and wrong destination port). Those scenarios are therefore not claimed as completed in this report.

9. Results Summary

Requirement	Result based on supplied outputs
Private DNS	PASS — app.team1.test resolved to 10.7.10.115 via 10.7.18.72
Backend A	PASS — direct HTTP 200 OK; X-Backend: A when running
Backend B	PASS — direct HTTP 200 OK; X-Backend: B when running
HTTPS through nginx	PASS — HTTP 200 OK using the supplied CA certificate
Load balancing	PASS — six requests alternated B, A, B, A, B, A when both backends were available
Cache headers	PASS — Cache-Control: max-age=60 and ETag observed
Conditional caching	PASS — exact If-None-Match request returned 304 Not Modified
Wireshark	PASS — DNS, backend TCP/HTTP, and TLS traffic evidenced by screenshots
Failure handling	PARTIAL — Backend A unavailable, Backend B unavailable, and nginx 502 observed; other guide-listed failures not evidenced

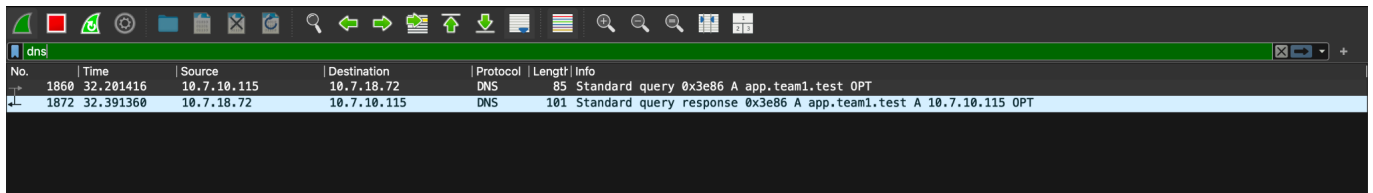
10. Conclusion

The two-Mac Phase 1 implementation demonstrates private DNS, two independently reachable Node/Express backends, nginx HTTPS termination and reverse proxying, load balancing, HTTP cache validation, and packet-level evidence. The recorded tests also demonstrate how nginx behaves when an upstream backend becomes unavailable and how it responds with 502 Bad Gateway when neither upstream is available. The implementation was restored to a working state in the supplied final checks.

Evidence basis: this report is organized around the two-Mac Phase 1 team guide and the Terminal/Wireshark outputs provided by the student. Where a required scenario was not evidenced in those outputs, it is explicitly marked as not demonstrated.

Appendix A — Wireshark Evidence

The following screenshots are included as visual evidence for the DNS, backend communication, and HTTPS/TLS tests described in the report.



The screenshot shows a Wireshark packet capture interface. The top toolbar includes icons for file operations, network analysis, and search. The packet list pane shows two packets. The first packet is a DNS query from 10.7.10.115 to 10.7.18.72. The second packet is a DNS response from 10.7.18.72 to 10.7.10.115, containing an A record for app.team1.test with IP address 10.7.10.115.

No.	Time	Source	Destination	Protocol	Length	Info
1860	32.201416	10.7.10.115	10.7.18.72	DNS	85	Standard query 0x3e86 A app.team1.test OPT
1872	32.391360	10.7.18.72	10.7.10.115	DNS	101	Standard query response 0x3e86 A app.team1.test A 10.7.10.115 OPT

Figure 1. DNS query and response for app.team1.test. The response maps the hostname to 10.7.10.115.

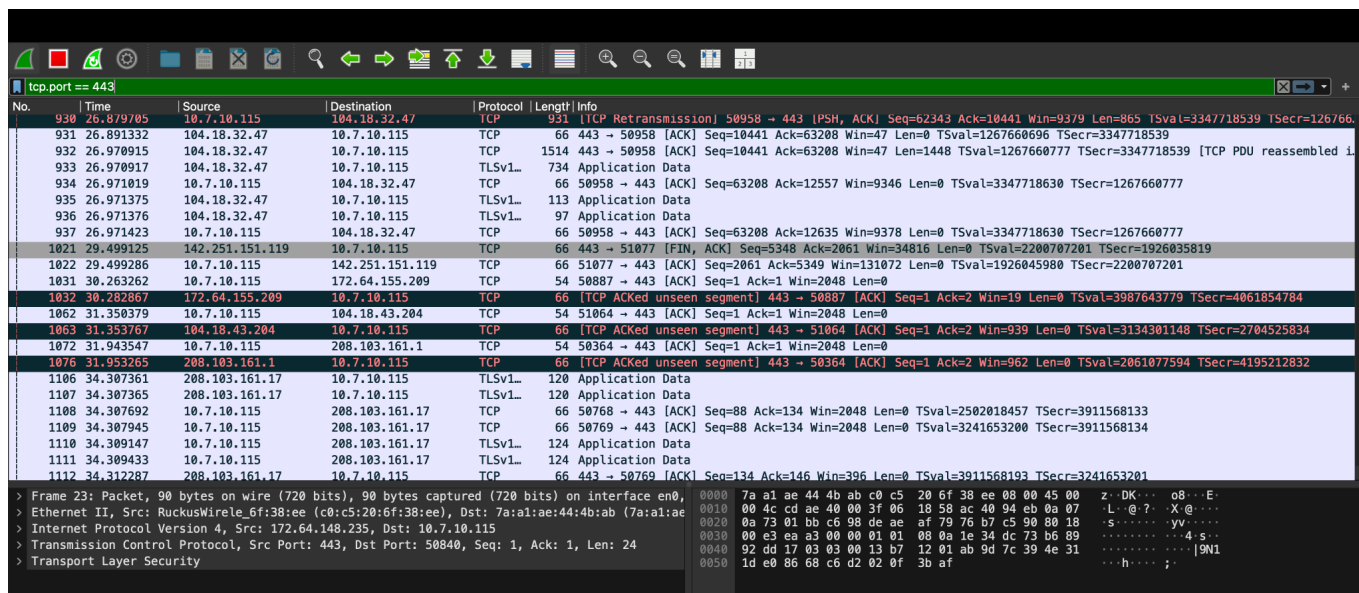


Figure 2. Backend B traffic on TCP port 3002, including the TCP handshake, GET /api/status, and HTTP 200 OK.

Capturing from Wi-Fi: en0

ip.addr == 10.7.18.72 && ip.addr == 10.7.10.115 && tcp.port == 3002

No.	Time	Source	Destination	Protocol	Length	Info
7918	204.969729	10.7.18.72	10.7.10.115	TCP	78	50753 → 3002 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TS
7919	205.018843	10.7.10.115	10.7.18.72	TCP	78	3002 → 50753 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 T
7920	205.019024	10.7.18.72	10.7.10.115	TCP	66	50753 → 3002 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=2396846705 TSe
7921	205.019191	10.7.18.72	10.7.10.115	HTTP	155	GET /api/status HTTP/1.1
7922	205.030278	10.7.10.115	10.7.18.72	TCP	66	3002 → 50753 [ACK] Seq=1 Ack=90 Win=131712 Len=0 TSval=2493520896 TS
7923	205.047581	10.7.10.115	10.7.18.72	HTTP/JSON	346	HTTP/1.1 200 OK , JSON (application/json)
7924	205.047697	10.7.18.72	10.7.10.115	TCP	66	50753 → 3002 [ACK] Seq=90 Ack=281 Win=131520 Len=0 TSval=2396846734
7925	205.048186	10.7.18.72	10.7.10.115	TCP	66	50753 → 3002 [FIN, ACK] Seq=90 Ack=281 Win=131520 Len=0 TSval=239684
7926	205.107739	10.7.10.115	10.7.18.72	TCP	66	3002 → 50753 [ACK] Seq=281 Ack=91 Win=131712 Len=0 TSval=2493520923
7927	205.107742	10.7.10.115	10.7.18.72	TCP	66	3002 → 50753 [FIN, ACK] Seq=281 Ack=91 Win=131712 Len=0 TSval=249352
7928	205.108100	10.7.18.72	10.7.10.115	TCP	66	50753 → 3002 [ACK] Seq=91 Ack=282 Win=131520 Len=0 TSval=2396846794

Figure 3. TLS/HTTPS packet capture showing encrypted application traffic.